

ST0257 – Sistemas operativos

Proyecto 2: Paralelismo con bloqueos en un ambiente controlado

MBA, I.S. José Luis Montoya Pareja
Departamento de Informática y Sistemas
Universidad EAFIT
Medellín, Colombia, Suramérica

RESUMEN

En este proyecto, los estudiantes deberán diseñar e implementar un sistema que sea capaz de manejar múltiples hilos concurrentes, utilizando métodos de control de concurrencia para sincronizar sus acciones. Deberán identificar y evitar posibles abrazos mortales (deadlocks) a través del uso de las técnicas de concurrencia adecuadas. El proyecto pondrá a prueba su capacidad para aplicar conceptos de concurrencia y sincronización en un entorno de programación simulado buscando garantizar la eficiencia.

PALABRAS CLAVE

Procesamiento en paralelo, paradigmas de programación en paralelismo, eficiencia, procesos, virtualización, memoria.

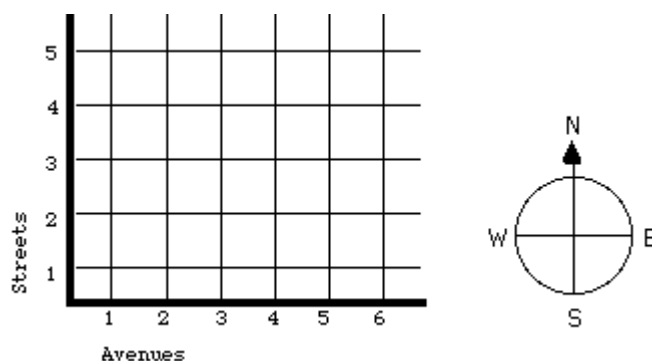
CONTEXTO

KarelJRobot

Este es un complemento que se monta en Java para tener un entorno donde visualizar el robot y poder programarlo. Lo vamos a utilizar para complementar los conocimientos en Java.

Conceptos básicos

KarelJRobot opera en un mundo que es una malla de calles y avenidas que permiten al robot moverse.



Las calles van de Este a Oeste y las Avenidas van de Norte a Sur. Tiene cosas especiales que el robot puede sentir y manipular. El mundo es esencialmente plano, empieza en la avenida 1 con calle 1 y puede extenderse hasta el infinito

dado que tiene al sur y al oeste muros infinitos que se extienden hacia el norte y hacia el este. El robot no puede atravesar esos muros dados que son de vibranio.

Las intersecciones entre las calles y las avenidas se denominan esquinas. En cada esquina se puede ubicar uno o varios robots, que tienen una brújula interna que les indica a donde están mirando.

Adicionalmente, podemos ubicar en una esquina una sirena (beeper). Las sirenas son unos pequeños conos plásticos que emiten un silencioso sonido de beep. Estas sirenas se pueden recoger, llevar y descargar en otra esquina o en la misma.

El robot es móvil, se puede desplazar en la dirección que está indicando su brújula (Norte, sur, este u oeste) y avanzan un paso a la vez. Pueden percibir eventos desde su entorno usando sensores rudimentarios de vista, sonido, orientación y tacto.

Los robots tienen una cámara que solo mira hacia adelante y está pensada para detectar un muro cuando está exactamente en frente de él (es miope el lente de la cámara). También puede escuchar una sirena cuando está parado encima de ella. Tiene un brazo que le permite recoger del piso las sirenas y colocarlas en su morral y extraerlas del morral y colocarlas en la esquina donde está parado. En el morral puede tener infinitas sirenas (como el morral de Dora, la exploradora). Los siguientes son los métodos disponibles para usar con el robot:

boolean anyBeepersInBeeperBag()

Determina si el robot tiene algún beeper en su bolsa de beepers.

boolean facingEast()

Determine si el robot está mirando hacia el Este o no.

boolean facingNorth()

Determine si el robot está mirando hacia el Norte o no.

boolean facingSouth()

Determine si el robot está mirando hacia el Sur o no.

boolean facingWest()

Determine si el robot está mirando hacia el Oeste o no.

boolean frontIsClear()

Determina si el frente del robot está libre o si está mirando un muro.

boolean nextToABeeper()

Determina si el robot está en la misma esquina de un beeper.

void move()

Mueve el robot al siguiente cruce dependiendo de donde esté mirando.

void pickBeeper()

Recoge un beeper de la esquina donde se encuentra el robot y lo lleva a la bolsa de beepers.

void putBeeper()

Coloca un beeper que se encuentra en la bolsa de beepers y lo coloca en la esquina donde se encuentra el robot.

void turnLeft()

Gira el robot 90 grados a la izquierda.

void turnOff()

Apaga el robot y lo coloca en color gris.

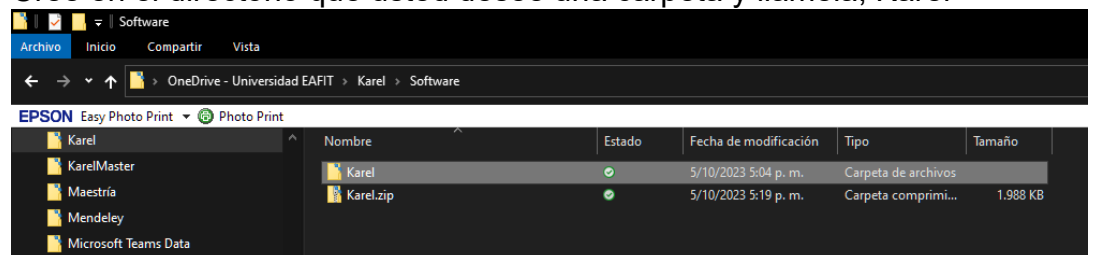
EJERCICIO 1: Instalación de KarelJRobot

Prerequisitos

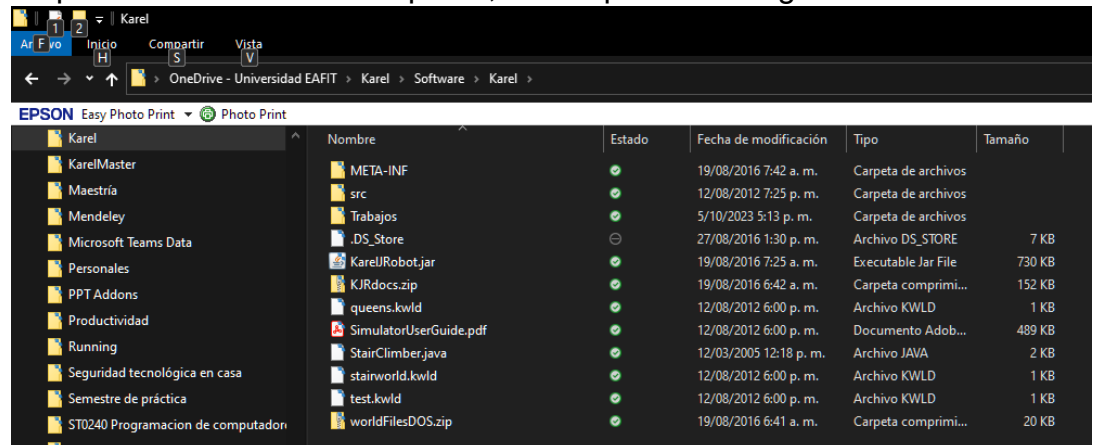
Debe tener la última versión del JDK (kit de instalación de Java donde está el compilador de java, javac). Los usuarios de MAC deben poder tener acceso vía consola.

Pasos

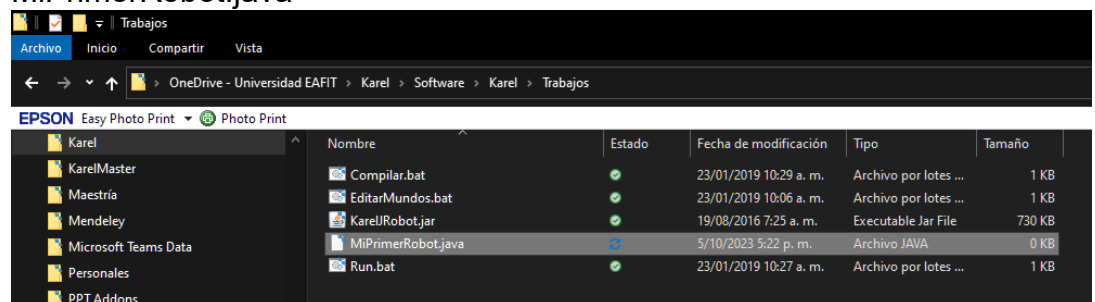
1. Vaya al Teams del curso, carpeta Materiales de clase y descargue a su máquina el archivo Karel.zip
2. Cree en el directorio que usted desee una carpeta y llámela, Karel



- Extraiga el contenido del archivo KJRDistribution160819.zip en dicha carpeta. Una vez lo descomprima, debe aparecer el siguiente contenido



- Vaya a la carpeta Trabajos y ahí cree un nuevo archivo Java llamado MiPrimerRobot.java



- Con el IDE que tenga en su máquina (Visual Studio o NetBeans o el que tenga) edite su programa con el siguiente contenido:

```
import kareltherobot.*;
import java.awt.Color;

public class MiPrimerRobot implements Directions
{
    public static void main(String [] args)
    {
        Robot Karel = new Robot(1, 1, East, 0);
        // Coloca el robot en la posición inicial del mundo (1,1),
        // mirando al Este, sin ninguna sirena.

        // Mover el robot 3 pasos, girar hacia el norte y apagar el robot.
        Karel.move();
        Karel.move();
        Karel.move();
        Karel.turnLeft();
        Karel.turnOff();
    }
}
```

6. Usuarios Windows: Graben el archivo y editen el archivo llamado "Compilar.bat". El archivo tiene el siguiente contenido:

```
javac -d . -cp ".;KarelJRobot.jar" %1
pause
exit
```

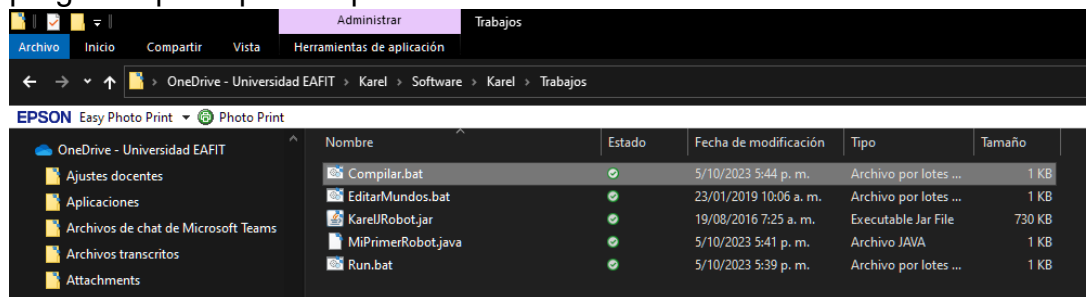
Reemplacen el %1 por el nombre del archivo Java (MiPrimerRobot.java)

```
javac -d . -cp ".;KarelJRobot.jar" MiPrimerRobot.java
```

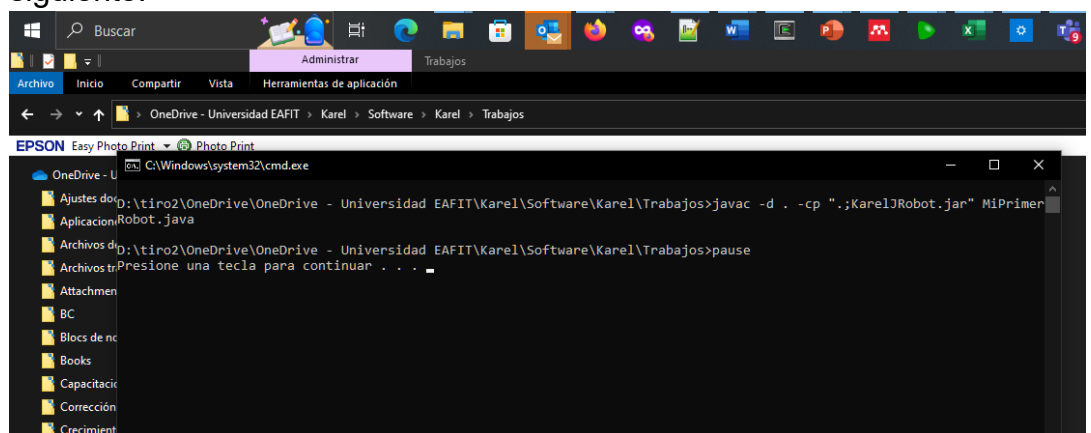
Usuarios MAC: Deben ejecutar en la carpeta y por la consola, el comando:

```
javac -d . -cp ".;KarelJRobot.jar" MiPrimerRobot.java
```

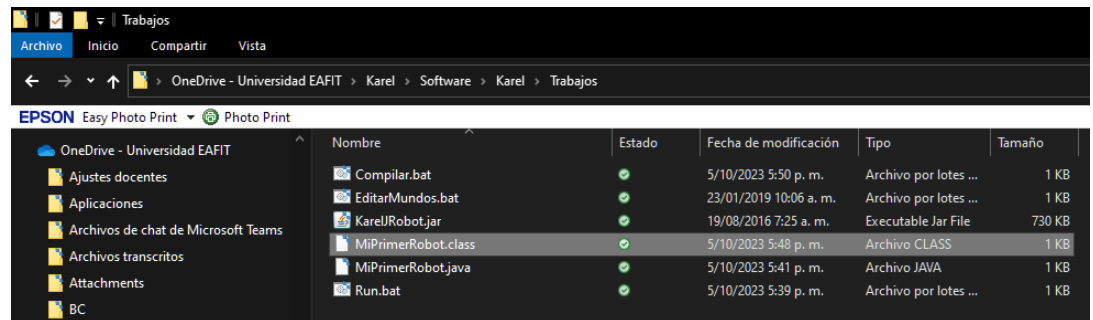
7. Guarden el archivo y en el Explorador de archivos, den doble clic al programa para que compile:



8. Si la compilación no tiene errores, debe salir una pantalla negra como la siguiente:



9. Presione cualquier tecla para cerrar la ventana negra. Luego revise que debe haber quedado el archivo MiPrimerRobot.class en la carpeta:



Si esto no sucedió, revise bien los archivos desde el paso 5 para que se aseguren que todo haya quedado como se pide en los pasos anteriores.

10. Usuarios Windows: Edite el archivo "Run.bat" el cual tiene el siguiente contenido:

```
java -cp ".;KarelJRobot.jar" %1
pause
exit
```

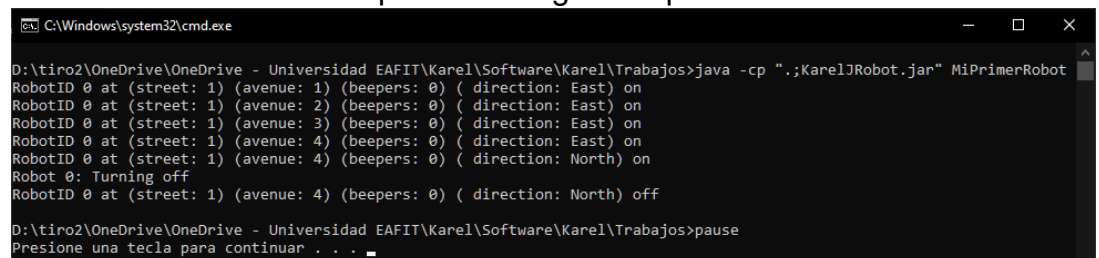
Cambie el %1 por el nombre de la clase (MiPrimerRobot), el archivo debe quedar así:

```
java -cp ".;KarelJRobot.jar" MiPrimerRobot
pause
exit
```

Usuarios MAC: en la consola y en la carpeta Trabajos, deben ejecutar el comando:

```
java -cp ".;KarelJRobot.jar" MiPrimerRobot
```

11. Guarden el archivo y en el Explorador de archivos dar doble clic en el archivo "Run.bat". Debe aparecer la siguiente pantalla:

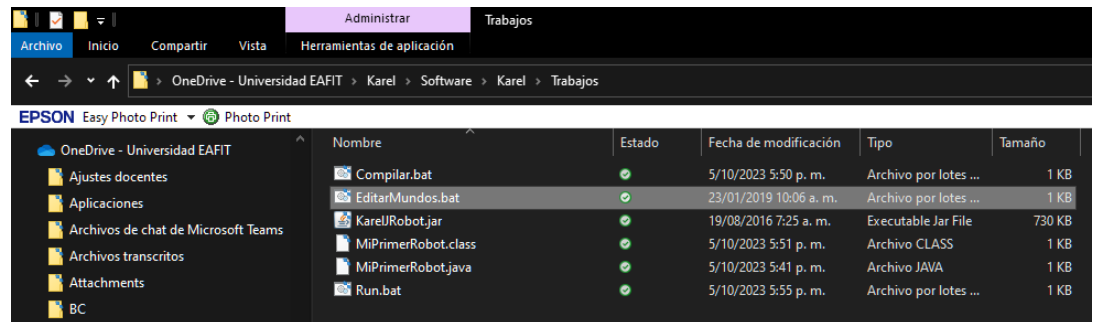


Esto significa que el programa corrió sin problemas. Si no aparecen estos mensajes, primero revise el archivo Run.bat del paso 10 y luego los pasos desde el 5.

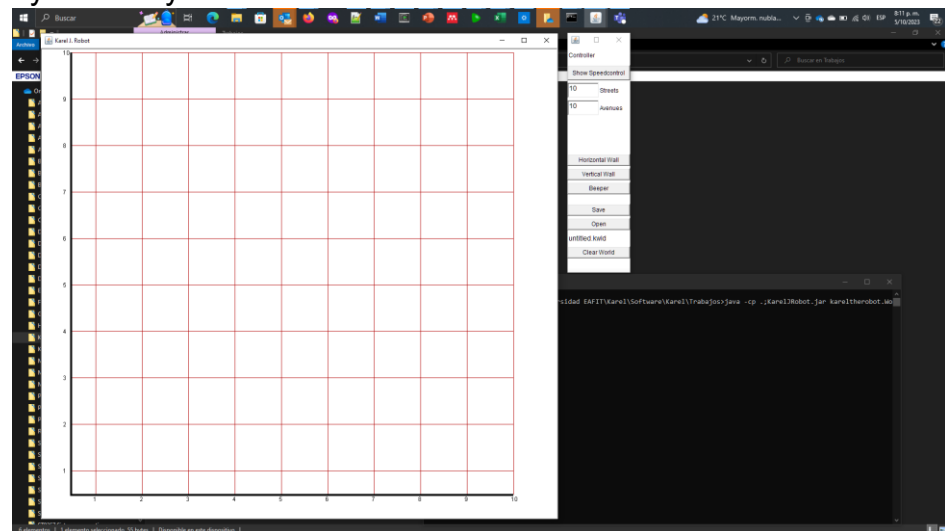
12. Tome un screenshot o una foto de la ejecución de su robot y póngala en la bitácora como evidencia del primer ejercicio.

EJERCICIO 2: Correr el robot visualizando el mundo

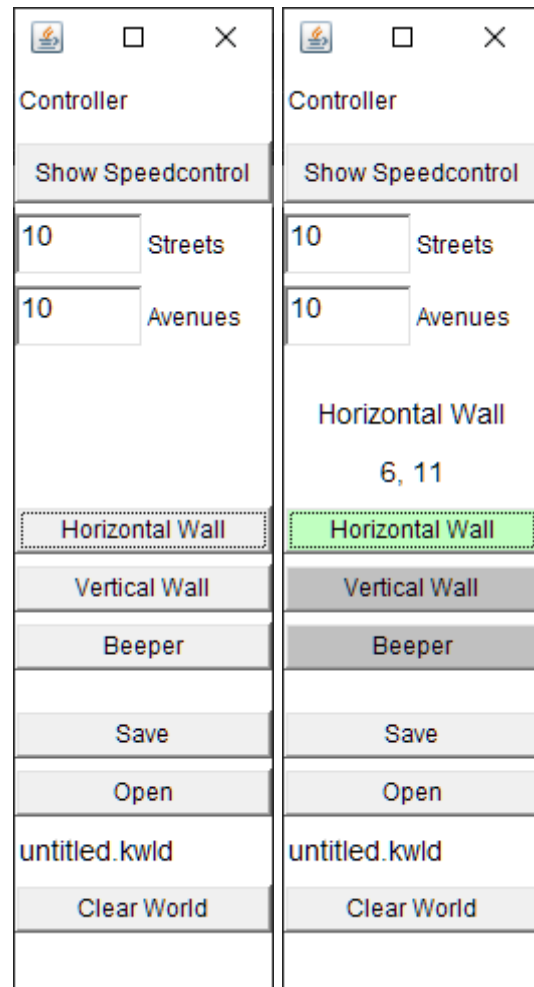
1. Use el editor de mundos para crear un mundo. Esto se hace dando doble clic en el archivo EditarMundos.bat



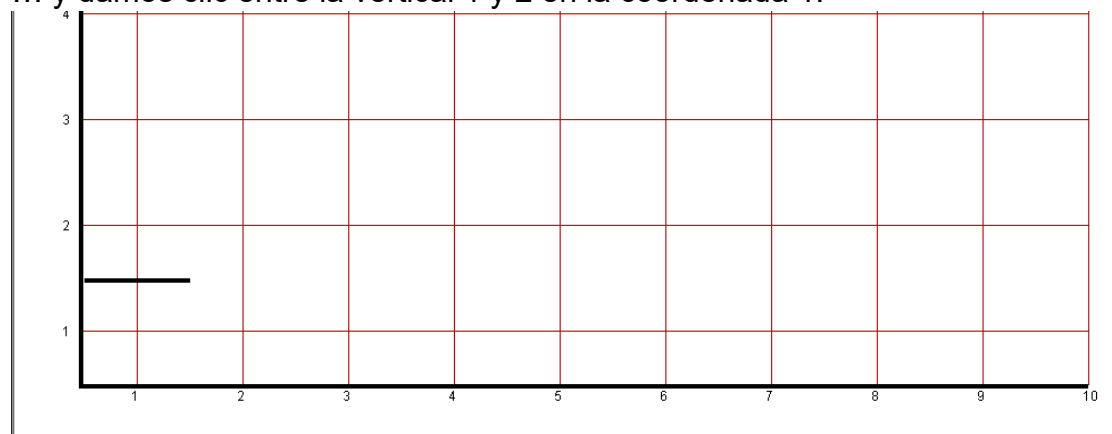
2. Aparecen tres ventanas: primero una negra donde aparecerán mensajes si hay errores y dos blancas donde se muestra el mundo del robot:



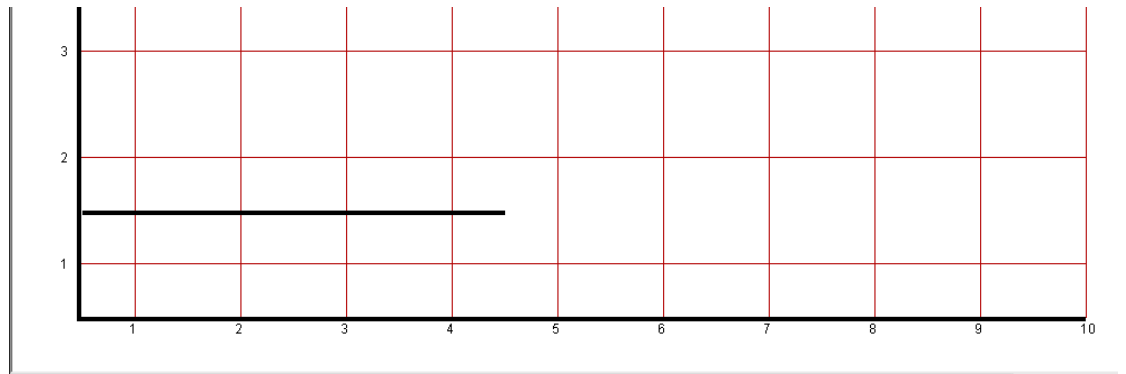
3. En esta interfaz se pueden crear los mundos. Vamos a agregar elementos al mundo. Empezamos con muros horizontales. Damos clic en el botón Horizontal Wall...



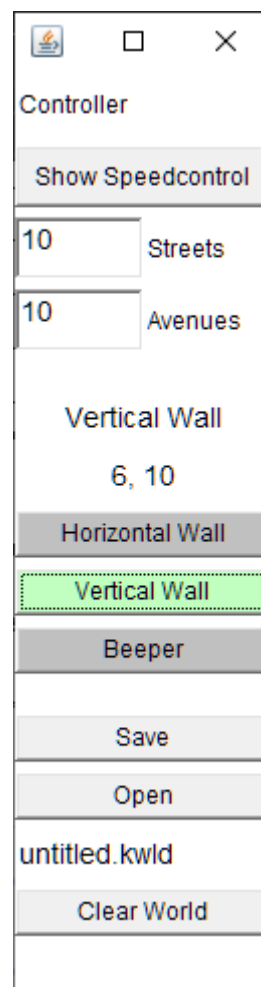
... y damos clic entre la vertical 1 y 2 en la coordenada 1:



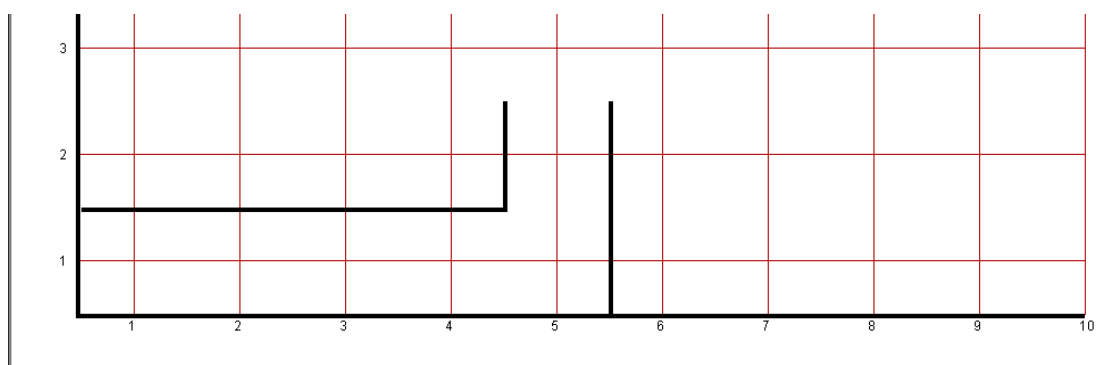
Repetimos lo mismo hasta la avenida 4.



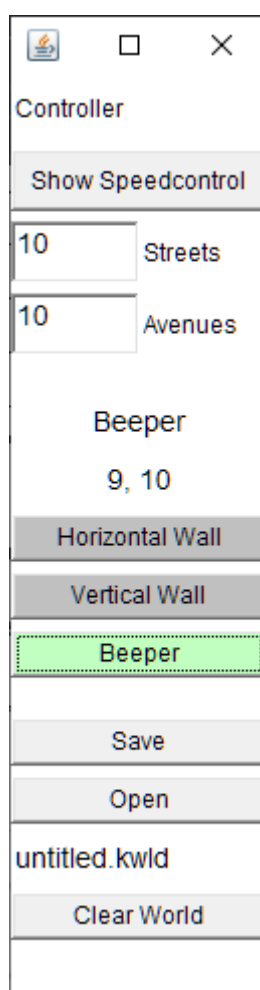
4. Ahora vamos a agregar muros verticales. Damos clic en el botón Vertical Wall...



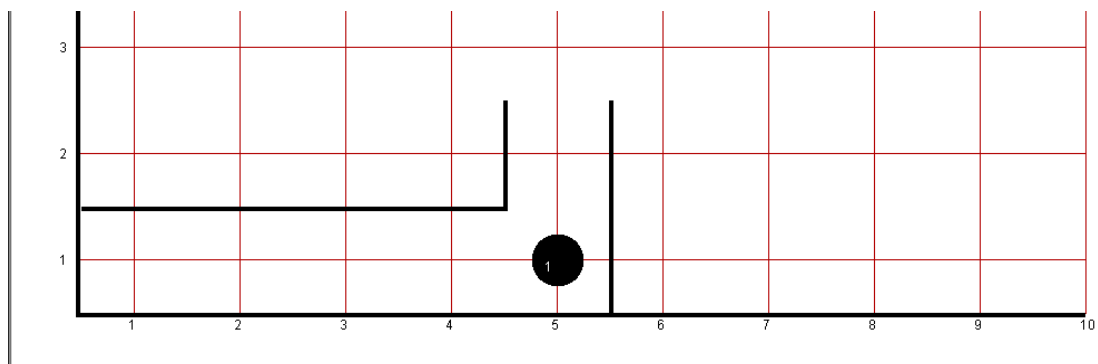
... y agregamos muros entre las avenidas 4 y 5 cruzando la calle 2 y entre las avenidas 5 y 6 cruzando las calles 2 y 1.



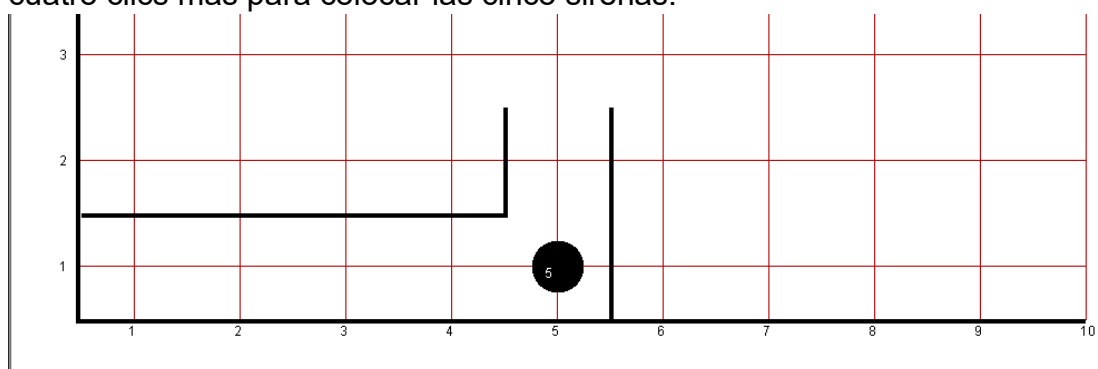
5. Por último, vamos a agregar 5 sirenas en la calle 1 avenida 5. Damos clic en el botón Beeper...



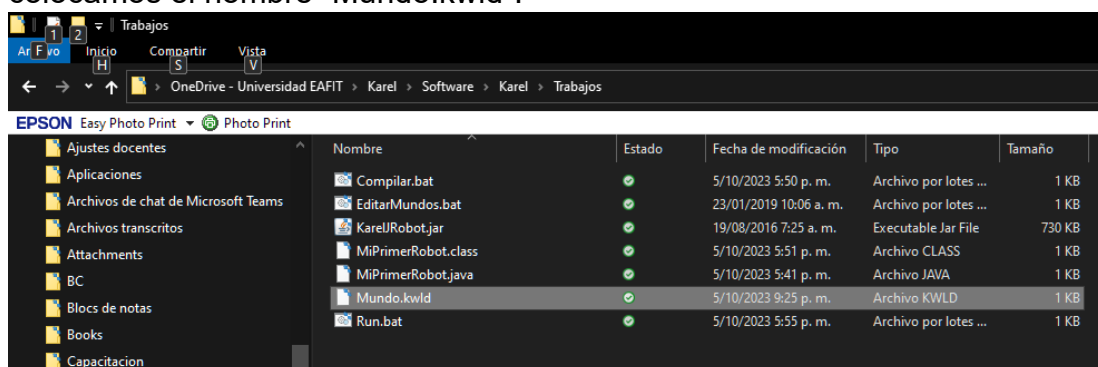
... y damos clic en la avenida 5 calle 1...



... como vemos, aparece un beeper (marcado con el número 1). Damos cuatro clics más para colocar las cinco sirenas.



6. Por último, guardamos el mundo dando clic en el botón Save. Le colocamos el nombre "Mundo.kwld".



7. Cerramos las ventanas y vamos al código para incorporar el mundo en el código. Debemos modificar el archivo MiPrimerRobot.java de la siguiente manera:

```
import kareltherobot.*;
import java.awt.Color;

public class MiPrimerRobot implements Directions
{
    public static void main(String [] args)
    {
        // Usamos el archivo que creamos del mundo
        World.readWorld("Mundo.kwld");
    }
}
```

```

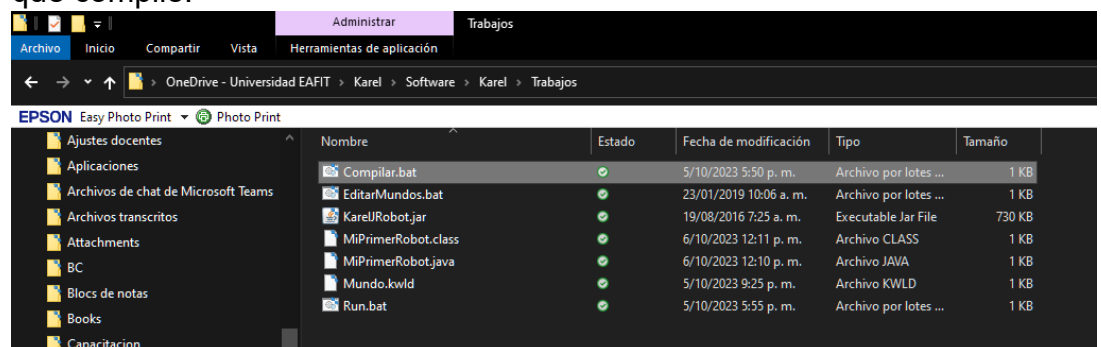
World.setVisible(true);

// Coloca el robot en la posición inicial del mundo (1,1),
// mirando al Este, sin ninguna sirena.
    Robot Karel = new Robot(1, 1, East, 0);

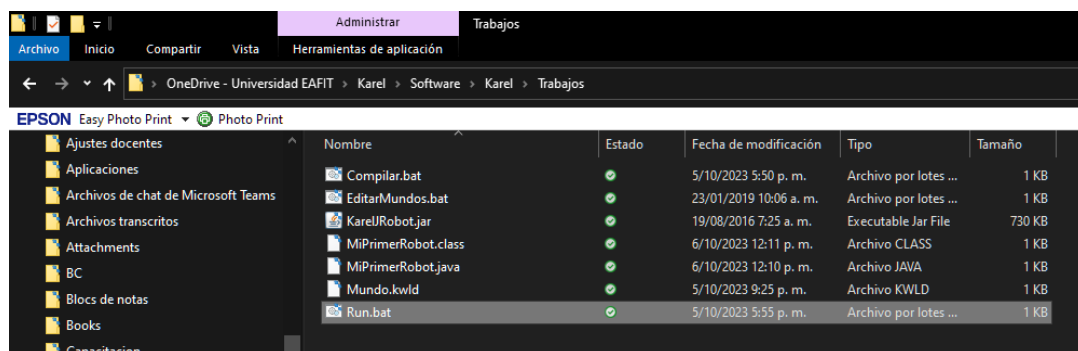
// Mover el robot 4 pasos
    Karel.move();
    Karel.move();
    Karel.move();
    Karel.move();
// Recoger los 5 beepers
    Karel.pickBeeper();
    Karel.pickBeeper();
    Karel.pickBeeper();
    Karel.pickBeeper();
    Karel.pickBeeper();
// Girar a la izquierda y salir de los muros
    Karel.turnLeft();
    Karel.move();
    Karel.move();
// Poner los beepers fuera de los muros
    Karel.putBeeper();
    Karel.putBeeper();
    Karel.putBeeper();
    Karel.putBeeper();
    Karel.putBeeper();
// Ponerse en otra posición y apagar el robot
    Karel.move();
    Karel.turnOff();
}
}

```

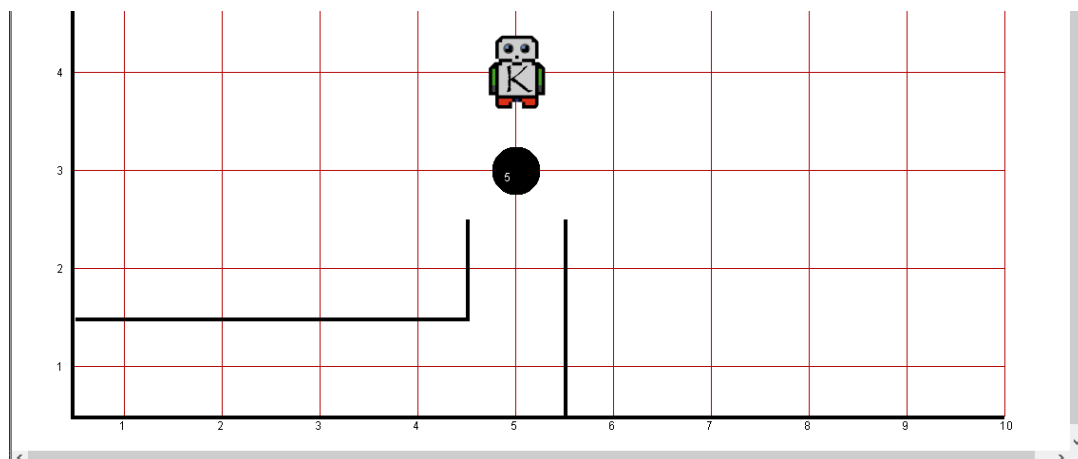
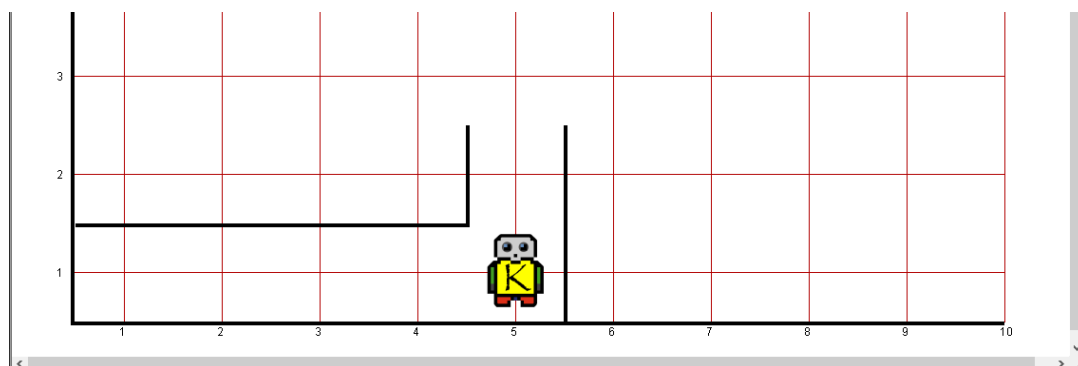
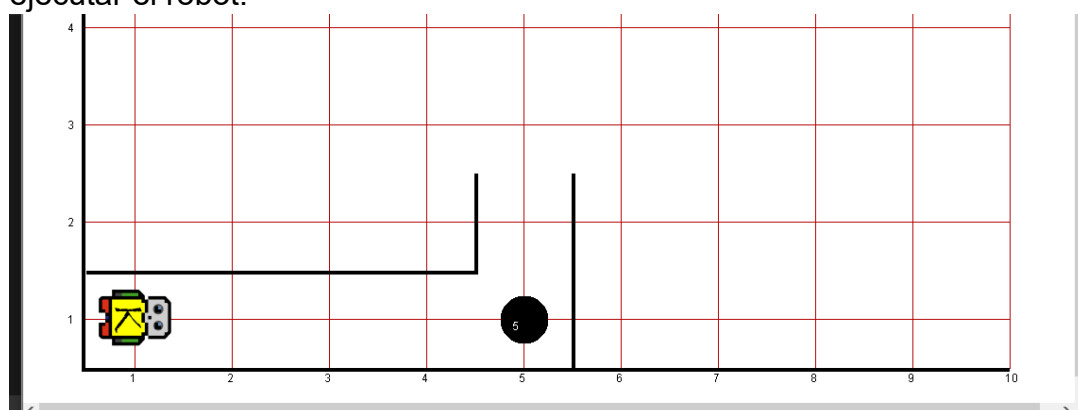
8. En el Explorador de archivos, den doble clic al programa Compilar.bat para que compile:



9. Y ejecutamos el programa dando doble clic en el programa Run.bat para ejecutar el programa:



10. Debe aparecer el mundo y ya el robot en la posición 1,1 y arranca a ejecutar el robot.



EJERCICIO 3

Prerequisitos

Ejercicios 1 y 2

Agregue al código del ejercicio 2, otro robot de color azul que comience en la misma posición del primer robot creado pero que se mueva uno después del otro (en el mismo hilo).

EJERCICIO 4

Prerequisitos

Ejercicio 3 y paralelismo en Java y en el robot.

Para que un robot pueda correr en su propio hilo, se debe crear una clase aparte que extienda la clase Robot:

```
class Racer extends Robot
{
    public Racer(int Street, int Avenue, Direction direction, int beeps)
    {
        super(Street, Avenue, direction, beeps);
        World.setupThread(this);
    }

    public void race()
    {
        while(! nextToABeeper())
            move();
        pickBeeper();
        turnOff();
    }

    public void run()
    {
        race();
    }
}
```

El método que activa el hilo es el run. En el ejemplo, éste llama al método race que es donde está el código que ejecuta el robot en su hilo. Luego, el main queda de la siguiente manera:

```
public static void main(String [] args)
{
    Racer first = new Racer(1, 1, East, 0);
    Racer second = new Racer(2, 1, East, 0);
}
```

Entonces solo es activar dos objetos de la clase Racer para que estos corran en hilos independientes y puedan realizar las tareas de manera independiente.

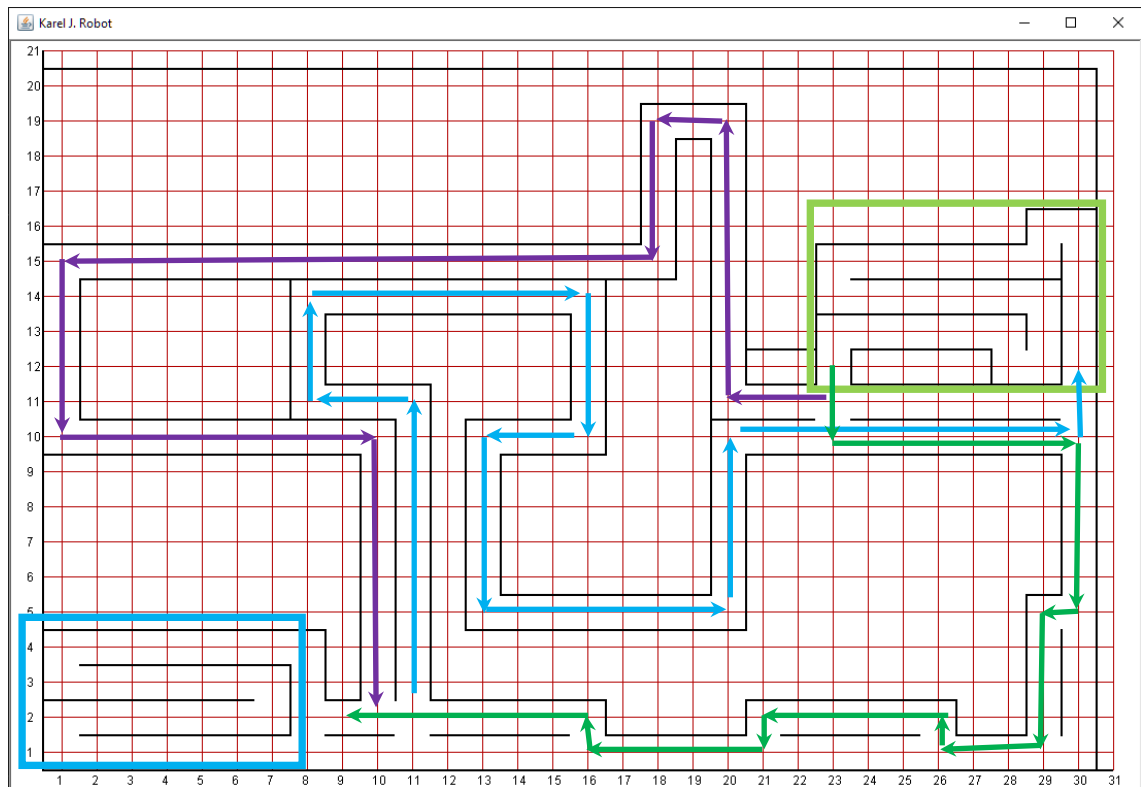
Programe dos robots según el ejemplo en el ejercicio dos para que hagan el mismo recorrido. El segundo robot debe tener color azul para distinguirlos.

Esto completará la primera parte de la práctica. La revisaremos en clase después de Semana santa.

Parte 2 – Carreteras y pare y siga

Una situación normal en las carreteras (sobre todo de Colombia) es que se presenten los famosos Pare y siga. Esta es una situación en la que la carretera solo tiene un carril de la vía disponible y alguien controla quien tiene la vía.

En el siguiente gráfico estará el mundo que representará esta situación:



El objetivo es llevar 500 pasajeros (beepers) desde la avenida 7 calle 1 hasta la avenida 30 calle 12 y 500 pasajeros (beepers) desde la avenida 23 calle 12 hasta la avenida 9 calle 2. Cada robot puede llevar hasta 4 pasajeros a su destino. Para hacerlo, hay dos posibles caminos.

Desde la zona azul:

1. El rápido pero con Pare y Siga: Saliendo desde la avenida 7 con calle 1, derecho hasta la avenida 30 calle 1 y de ahí subiendo directo por la avenida 30 hasta la calle 12.
2. El largo: en la avenida 11 calle 1, se gira a la izquierda y se va directo hasta la calle 11 y se sigue la ruta de flechas moradas.

Desde la zona verde:

1. El rápido pero con Pare y Siga: Saliendo desde la avenida 23 con calle 12, sale recto y gira a la izquierda hasta la avenida 30 calle 10, gira a la derecha sigue entrando en las bahías por la ruta verde hasta dejar los pasajeros en la avenida 9 calle 2.
2. El largo: Se sale desde la avenida 23 con calle 12 y se gira a la derecha en la calle 11 y se sigue la ruta negra.

La razón de tener dos caminos es el Pare y Siga. Si observan bien el camino rápido, hay que parar en las bahías mientras pasan los robots en el otro sentido. Si hay cuatro robots entre las avenidas 12 y 15 de la calle 1, necesariamente deben girar a la izquierda por el camino largo los robots que salgan hasta que avance al menos uno de dichos robots. Por otro lado, si hay 6 robots en la calle 10 entre avenidas 24 y 29, necesariamente el regreso debe hacerse por el camino largo hasta que se libere al menos uno de dichos robots.

Reglas

1. No se permite que dos robots estén en la misma posición del mundo. Si esto se da, es un choque.
2. En la zona azul y en la zona verde aparecen 28 robots en cada una.
3. Cuando llega a una estación, cada tren pasa 3 segundos antes de partir a la siguiente estación.
4. En el camino rápido se debe respetar los pare y sigas. Pasan de cuatro en cuatro hasta que cruzan al otro lado.

CONSIDERACIONES GENERALES

1. El desarrollo de la práctica puede ser individual o en equipos de máximo tres personas.
2. El desarrollo completo es en Java.
3. La práctica se entrega subiendo los fuentes en archivos (no comprimidos) y el informe por el buzón recepción de trabajos de Eafit Interactiva (cualquier otro medio no será admitido).
4. Cada semana se sacará un espacio de 10 a 20 minutos al inicio de la clase para hablar de la práctica y resolver dudas.
5. Criterios de evaluación (ver Anexo 1)

FECHA DE ENTREGA

Viernes 26 de septiembre en clase a través de Eafit Interactiva.

SUSTENTACIÓN

Viernes 26 de septiembre en clase. El mecanismo de sustentación es el siguiente:

1. Cada equipo muestra su desarrollo ejecutando en vivo, dejando ver que el programa se ejecuta con los tres modos solicitados.
2. Debe mostrar cómo solucionó cada uno de los retos y cómo lo relacionaron con conceptos vistos en clase.
3. Mostrar los resultados y explicar las conclusiones.
4. Se realizarán preguntas por parte del docente para validar el entendimiento individual de los conceptos aplicados. Esto significa que, aunque el trabajo es en equipo, la nota del trabajo puede ser diferente para cada uno de acuerdo con la calidad de las respuestas.

Nombre de la asignatura: Sistemas Operativos

Competencia a la que aporta la asignatura: Conocer el sistema operativo del computador para un mejor desarrollo, diseño y ejecución de las aplicaciones y aplicar nuevas soluciones.

Resultado de asignatura evaluado: Simulación de transporte.

Evento evaluativo: Proyecto 2

% del evento evaluativo: 30%

Criterios (que tributen al RA de asignatura)	Cumple con altos estándares (4.5 -5)	Cumple a satisfacción (4 -4.4)	Cumple parcialmente (3.5-3.9)	Incumple parcialmente (2.5-3.4)	Incumple totalmente (0 -2.4)	Peso asignado al criterio sobre la calificación.
Análisis de fundamentación para la creación de la biblioteca. Claridad en el concepto	Entiende completamente la necesidad y plantea varias opciones de solución. Utiliza conceptos adecuadamente para la solución.	Entiende completamente la necesidad y plantea una opción de solución.	Omitió un elemento clave para el entendimiento de la necesidad	Omitió varios elementos para el entendimiento de la necesidad.	Demuestra poco o nulo entendimiento del problema.	40%
Diseño de la solución Solución óptima	El diseño tiene en cuenta los conceptos vistos en clase y argumenta la elección de su solución. La solución elegida es la óptima para el problema.	El diseño tiene en cuenta los conceptos vistos en clase y argumenta la elección de su solución.	Aunque se tuvieron en cuenta conceptos vistos en clase, no hubo argumentación correcta en la elección de la solución.	No se tuvieron en cuenta los conceptos vistos en clase.	Demuestra poco o nulo entendimiento de patrones de paralelismo al momento de explicar la solución.	40%
Funcionalidad Calidad de la solución frente a necesidad planteada	El programa funciona correctamente y se entrega de la forma descrita en el documento.	La solución elegida no es la óptima pero el programa funciona correctamente y se entrega de la forma descrita en el documento.	El programa con los conceptos vistos en clase no funciona correctamente o no se entrega de la forma descrita en el documento.	El programa no tiene en cuenta ningún concepto visto en clase y la solución funciona correcta o parcialmente o no se entrega correctamente.	Se entrega la solución parcialmente o no se entrega ninguna solución o no se entrega correctamente.	20%